

Active InferAnt Stream 014.1 ~ Generalized Notation Notation: From Plaintext to Triple Play

ActiveInferAntStream | Generalized Notation Notation: From Plaintext to Triple Play | 1:45:29

all right hello welcome everyone it's May 15th 2025 this is active in for ant stream 14.1 and we're going to begin it as we do with a github push and we're going to go from there okay this stream is called generalized notation notation from plain text to triple play and before jumping into the background want to give a little bit of the payload up front here's the simple structure of the repo and where we're headed all the source code is in src all the documentation is in doc and all the output from running main goes to output so what is the output going to look like well first where does it start from and where is it going what it starts from is looking like a markdown file with a controlled vocabulary describing state spaces for a given generative model and here's where we take that first for each of those provided source files gnn files we get a bunch of visualizations so anyone who's worked with pymdp or active inference models knows that figuring out state spaces and doing all kinds of work and transformations with state spaces can be challenging so one thing this does is it takes that generative model distinction as specified and visualizes it it outputs it exports the model into a variety of formats json text xml graphical all these different kinds of outputs there's a initial processing report essentially summarizing what kinds of files are being detected there are rendered simulations so here is a pymdp script that fills in not perfectly functionally at this point but i think more than enough to get the picture a pymdp script parameterized by the details of the state space of the source file and where this is all going is to be able to cross render into multiple implementation strategies so a stub method for going into rx infer type checking is performed so that is checking the validity of the code and making sure that the source files are valid and this allows us to bring in not just security features but also something that is often much desired for those running active inference models which is the computational resource estimation so here we can look at the state space sizes and the different kinds of operations that are entailed for a given generative model and then get different kinds of assessments like what is the relative computational cost going to be or what is going to be the size for a given number of variables of storing that specific model on on the ram or on the hard drive uh llms are used at the end to in this case summarize a set of questions so here are uh reports written with llms summarizing the states based model the logs for improved tracking and coding mcp is the model context protocol and we'll get into this a little bit more but this is a very large file which concatenates all of the mcp methods which we're exposing like visualize gnn file

so this is something that you can run locally
or we can stand up and we will stand up mcp servers
so that these kinds of services can be provided
ontology processing
these are summaries of the ontology terms used
and how they map onto different state spaces
and this is really one of the key pieces of gnn
which is putting a little bit of space between
putting a flexible interface
and gap between
a given variable
and its ontological annotation
so
commonly in looking at these generative models
cognitive modeling type scripts
you'll see the name of a variable
like observation modality 1
or latent state
or whatever it happens to be
some english language name of a variable
however
that can get into some issues
because
what if it's modality 3
but then one of the other ones comes and goes
or what
how do you add modality 4
or it's the sense variable for one agent
but it's the action output from another agent
so in other words
you don't want to have
the variable
always welded to its ontological assertion
because there can be multiple
english terms
or roles
for a given state space variable
we're going to come back to that with cerebrum

and then also there can be different languages
for localization and accessibility
so
there's a space between
the variable state space
and the assertion
and what that looks like
inside of the GNN file
is a specific section
that makes assertions
about ontological meaning
in that exact generative model
so
gone already
long long long gone
are the days where
debating whether
the letter b
means
transition matrix
well
no
it's just a symbol
that in a given generative model
might be asserted to mean transition matrix
then
those rendered
scripts
so we had the program synthesis
of the pymdp agent
parameterized from the GNN file
and then we can actually execute those
and
this one
hit an error
that you'd hit
running pymdp
and so did this one

but in previous versions
I've had it running
so it's just
in and out
all presented
as open source
work in progress
and
inserting
the correct value
values
into
the
stem
and stub
of a pymdp script
is totally doable
and it will be coming
when the time is right
then there are test reports
so this is just
a test
information
from a test
module
and
overall
a processing
summary
which has been improved
and
disimproved
and all of that
okay
so that's where we get
with the outputs
how is this all
gonna be happening

let's pull back a second
if anyone has any comments
or questions
please write them
in the live chat
all this
is open source
at the
github repo
active inference institute
slash
generalized notation
notation
no spaces
I'm using
the updated version
of cursor
0.50
0.4
and the
large language model
for most of the
augmented coding
5 coding
happening here
is gemini 2.5 pro
the
preview
05
06
so just from
a few
days ago
okay
we will
briefly
look at
the source

structure
get a first pass
pull back a little bit
to the history
of gnn
then re-enter
into the source
with people's comments
and questions
so
here's how
source
folder
looks
okay
source
folder
has a
main
script
so when you want to
replicate
the entire
process
you just go to
src
and run
python3
main
so there's a single
entry point
that is going to
look like this
it's going to start
go through
1
which is just confirming
that it has the

gnn payload

go into

2

which is the

setup phase

that's going to

install the

requirements

any other packages

that you add in

requirements.txt

it is going to go

into

3

which is the

test suite

there's just a few

tests right now

but you can add more

tests

4

goes into the

type checker

suite

which has the

type checking

as well as the

resource estimation

methods

after 4

it will head into

5

the export module

and so the

folders

are not numbered

but the scripts

are just to

really confirm

here's the order

that we're running

these different

modules in

5

goes into

the export

so you can

flag different

export modalities

6

into the

visualization

methods

7

is the

MCP

the model

context

protocol

section

which we'll

go into

more detail

8

ontology

terms

ontology

analysis

9

is the

rendering

so that's

like the

setup

of the

simulation

that's the
transformation
from the
generic
GNN form
and parsed
output
into a
rendered
but not
executed
e.g.
pi mdp
rx infer
ngc learn
etc
etc
etc
we'll build
those adapters
and they'll be
really robust
step 10
is running
the
simulation
so actually
this one
let's see
if it
came
through
this is
looking like
it actually
did do the
pi mdp
simulations

so let us
see
slow computer
there we go
so
not
exactly
fully
functional
but this
is
demonstrating
that
we can
do
program
synthesis
controlled
vocabulary
program
synthesis
in an
implementation
agnostic
fashion
with static
checking
render it
into a
given target
implementation
style
like
pi mdp
and then
run
that
pi mdp

method
and
it
looks like
there still
could be
some
errors
in the
simulation
but
it is
breaking
on through
to the
other
side
that's
10
then
11
runs
the
llm
section
which
you can
customize
and it's
going to
go into
this
llm
processing
and that's
where it's
going to
do things

like
summarize
the
GNN
file
or do
a set
of
questions
that are
responded
to
by the
llm
and give
an executive
summary
then
it's going
to finish
the 11th
one
and give
you some
summaries
on the
whole
pipeline
so
again
just while
we're
overviewing
the source
folder
structure
and then
heading

into the
context
and the
history
of
GNN
main
is going
to call
the 11
scripts
and you
could just
start up
another
prompt
make me
a 12th
script
and a
folder
for doing
dot dot dot
like if
somebody writes
a question
or a comment
in the live
chat
I very
may
well
add another
module
or do
some
other
method

then
each
of these
scripts
which are
run by
main
are going
to look
into the
folder
of their
own
name
so for
example
the type
checking
is going
to look
into
GNN
type
checker
and it's
going to
find
different
files
like the
resource
estimator
methods
all of
that
and so
you can
choose

in this
sort
of
main
first
shell
layer
of
dispatch
which
tools
you want
to bring
into
play
and then
we can
advance
out the
tech
tree
in a
really
modular
way
like
okay
we have
type
checking
on
models
with
planning
but
we
don't
have

py
mdp
for it
and then
we'll
just
go to
the
script
where
it's
working
and then
reverse
engineer
what is
the
state
space
specification
we need
to do
so that
we can
generate
that
script
that
did
work
and
if
we
can
make
that
connect
we're

going
to
be
able
to
have
a
ton
of
flexibility
and
a
lot
of
reproducibility
and
integrity
for
the
models
that
we
run
another
feature
that's
going
to
come
into
play
in
the
mcp
module
is
that
each

module
has
an
mcp
dot
py
script
and
these
are
going
to
be
methods
that
become
available
to
the
mcp
module
so
the
mcp
module
when
it
is
run
is
going
to
look
in
the
src
folder
and

say
okay
go
into
all
the
folders
and
find
the
mcp
py
and
then
pull
all
the
methods
from
the
mcp
py
and
that
will
get
reported
on
in
the
mcp
processing
step
which
is
where
we
get

these
very
very
information
dense
super
informative
very
information
rich
files
about
which
methods
we're
exposing
in
this
implementation
so
then
if
you're
running
a
version
and
you
don't
have
the
execute
or
you
have
a
certain
version

of
one
of
these
or
a
different
folder
as
long
as
you
have
that
mcp.py
it's
going to
control
which
services
you're
giving
model
context
protocol
availability
to
locally
or
remotely
and
so
there's
some
super
fun
ways
to

think
about
that
from
like
a
business
slash
institution
perspective
like
if
it's
worth
one
cent
to
verify
a
generative
model
or
to
render
it
into
py
mdp
or
to
execute
it
then
maybe
with
micropayments
those kinds
of things

will become
easier
to do
like
x402
which
holly
shared
in a
meeting
yesterday
and
big
picture
on
mcp
this
is
an
example
of
a
neurosymbolic
architecture
because
there
can
be
generalist
agents
multimodal
agents
whether
they're
probabilistic
models
like
neural

networks
transformers
llms
etc
or
it's
a
person
with
a
pen
and
paper
index
card
by
opening
up
this
GNN
stack
as
an
mcp
nexus
we're
in a
new
space
where
the
llm
can
do
program
synthesis
and
execution

on
generative
models
so
it can
be
using
its
own
weights
its
own
inference
methods
to
do
certain
things
including
tool
switching
and
then
if
there's
a
given
situation
where
explicit
planning
is
needed
it's
not
that
we
need

the
llm
or
the
neural
network
to
do
explicit
planning
but
it
could
spin
up
a
simulation
and
dispatch
the
execution
that
does
do
explicit
planning
so
like
if
you
go
to
an
llm
and
you
say
hey

what's
the
expected
free
energy
of
me
doing
this
on
this
day
it
might
give
you
a
number
or
it
might
give
you
some
relative
variable
and
that
might
be
directionally
accurate
it
might
be
valid
might
be

good
enough
so
it's
not
to
say
that
it's
not
but
what
it's
not
doing
is
doing
program
synthesis
on a
generative
model
and
then
using
free
energy
minimization
to
actually
work
on
that
model
and
leaving
a
full

type
checked
trace
all
along
the
way
so
that's
the
sort
of
neurosymbolic
programming
aspect
that is
going to
be super
interesting
I
always
think
of
this
like
a
chocolate
chip
cookie
where
the
soft
cookie
part
is
like
the
generalist

probabilistic

agent

and

then

the

chocolate

chips

are

like

the

GNN

or

the

tools

that

it

can

call

through

MCP

or

through

other

techniques

so

we

now

have

really

good

dough

better

and

better

and

really

good

chocolate

fragments
better
and
better
too
so
there's
a lot
of
opportunities
for
this
kind
of
crystal
embedded
in
a
more
liquid
or
gelatin
like
interface
that
is
going
to
enable
models
again
that
don't
have
explicit
planning
or
other

features
like
that
to
drop
in
and
deploy
these
actual
simulations
with
really
strong
reproducibility
and
tracing
okay
so
where
did
this
GNN
concept
come
from
meanwhile
of
course
anyone
can
write a
comment
or a
question
and
I'll
hope

to
bring
some
new
features
in
to
the
package
like
by
the
end
of
the
stream
so
this
work
was
during
2022
and
2023
working
with
Jakob
Smekl
who's
currently
a graduate
student
at
Stanford
we
were
doing
various

things
with
multi-agent
systems
and
having a lot
of fun
talking about
active
inference
and AI
and
whatnot
and
we
started
to
explore
this
idea
of
making
notation
systems
that
were
implementation
agnostic
for
active
inference
and
using
that
for
our
own
epistemic

and
pragmatic
ends
so
for
example
in
the
step-by-step
active
inference
paper
of
Smith
Friston
and
White
from
2022
this
is
like
a
common
step-up
in
terms
of
learning
about
generative
models
so
like
in
the
top
left

there's

D

which

is

a

prior

on

latent

states

S

the

latent

state

itself

like

the

temperature

in

the

room

O

the

observation

on

the

thermometer

and

then

A

the

mapping

between

the

real

temperature

in

the

room

and
the
thermometer
so
it's
a
static
one
time
step
perception
you've
got
a
prior
on
temperature
a
thermometer
reading
coming
in
and
then
through
the
ambiguity
matrix
A
you
come
to
some
posterior
on
latent
state
then

you
can
add
one
more
motif
in
with
a
B
transition
matrix
then
you
can
have
temperature
changing
through
time
emitting
observations
with
thermometer
readings
this
is a
passive
inference
it's
a
partially
observable
Markov
process
then
that
can

get
bumped
up
to
a
partially
observable
Markov
decision
process
in this
case
where
the
decisioning
is
represented
by
this
pi
policy
vector
of
affordances
which
are
operated
on
by
an
expected
free
energy
functional
which
intakes
a
C

preference
vector
and
then
that
can
even
be
extended
further
to
have
this
E
habit
vector
and
this
precision
temperature
shaky
hand
on
action
stuff
happening
so
it's
like
okay
how
do
we
really
look
at
that
though

and
understand
ideally
in an
open
source
plain
text
way
really
get
down
to
what
are
the
specific
similarities
and
differences
between
these
models
and so
that led
to us
which we
jokingly
called
land
gauge
because of
gauge
theory
where
these
are
plain

text
and you
can
take
a
given
column
from
the
coda
and you
can
just
paste
it
into
a
notepad
and it
will
come out
like a
markdown
and it
has different
sections
and it's
a pretty
similar
structure
today
two years
later
looking
in the
GNN
examples
looking here

we see
some of
those
similar
fields
and a
similar
state
space
notation
as to
what it
was
where you
declare
the state
space
you
declare
the
connections
give
initial
parameterization
add any
other
equations
how is
time
treated
what is
the
annotation
of the
ontology
and then
we could
have other

sections
like what's
a footer
or we
could have
a cryptographic
signature
so that
was version
one of
the paper
in the
earlier
days of
active
inference
institute
fun
where we
wanted to
highlight
that
we could
bring together
all these
different ways
of thinking
about program
synthesis
and we
could express
it in
what is
also the
title of
the stream
which is
the triple

play
so
the triple
play
is
the high
fidelity
rendering
of a
GNN
expression
whether
it's a
small
motif
or whether
it's a
whole
ecosystem
scale
multi-agent
generative
model
hybrid
synthetic
intelligence
etc
etc
etc
across
and here's
the triple
play
plain text
graphical
and executable
computational
forms

moving in
and out
of these
forms
and that's
the triple
play
so then we
talk a little
bit about
those three
aspects of it
the text-based
model
plain text
benefits of
plain text
graphical
models
here of course
playing on the
double play
of graphical
itself
being like a
graphical
relational
model
but also
being visual
diagrammatical
graphical
and then the
executable
cognitive
models
so sometimes
you just

want to
like
dream a
big dream
about a
recipe
but there's
recipes you
can specify
that you
can't cook
at home
you just
don't have
the ingredients
or no
kitchen could
possibly do
this or
that
so separating
out
where are
we just
writing
creative
speculative
semi-fiction
generative
model play
when are we
visualizing
the aforementioned
and then when
do we really
want to execute
it and when
it gets down

to executing
it just
like software
can be
implemented
on different
operating
systems
we want
to be
able
for flexibility
and reproducibility
to execute
cognitive models
in different
settings
so that
was our
paper
we
summarized
that
shortly
and then
we provided
a quick
assessment
on how
this GNN
is actually
not just
useful
and pragmatic
it's also
very epistemic
so we
dive a little

bit into
the realism
and instrumentalism
topic
and then
we talk
about how
we will
be able
to
implement
this triple
play
and here
we are
in that
ballpark
the very
same
implementing
the triple
play
and
even
planting
some
seeds
that
bloomed
many
many
April
showers
and
May
flowers
later
like

from a
linguistics
perspective
it would
be
interesting
to explore
possible
grammatical
case
systems
morphology
and dialects
associated
with GNN
it's not
just a
throwaway
line
that's
what
cerebrum
is about
case
systems
for generative
models
so now
we can
say
we want
the
locative
version
of that
GNN
and so
for those

who are
wondering
where does
cerebrum
come into
play
how can
I use
what I've
learned from
Sanskrit
and Russian
and Latin
and is
invisible
in English
how can
I use
that kind
of
morphological
morphosyntactic
understanding
of language
how can
I bring
that
to
high
reproducibility
generative
modeling
here is
how
and that
was the
paper
in 2023

with
Jakob
and
put it
on the
back burner
for a
little bit
but
now
the
repo
is
back
with
a
fury
but not
an angry
one
and
hopefully
this is
going to
be
a
really
key
tool
and
platform
for people
who want
to
use
generative
models
for

education
and
for
research
and
it
can
be
under
the
hood
it's
not
that
every
single
person
may
be
in
this
exact
repo
but
as
we
start
to
think
through
these
things
as
a
community
it
becomes
a lot

clearer
and
clearer
how
having
these
methods
for
setting
up
workspaces
testing
our
environment
type
checking
and
resource
estimation
exporting

methods
available
for
LLMs
and
other
agents
understanding
the
ontological
annotations
of
state
spaces
preparing
simulations
through

rendering
executing
the
simulations
and
using
LLMs
to give
interpretability
in natural
language
and do
other
kinds
of
operations
like
these
are
some
of
the
core
activities
that
we
need
for
our
own
research
work
so
it's
cool
that
we
can

develop
this
and
make
this
an
open
science
space
and
welcome
participation
from
people
who
see
opportunities
and
challenges
here
so
let's
do a
few
let's
check
out
the
agenda
and
see
where
we
might
want
to
do
something

and
then
again
if
somebody
is
watching
live
they
can
write
a
comment
or
an
idea
in
the
chat
so
what
is
GNN
and
why
is
it
important
GNN
is
a
plain
text
notational
style
and
tradition
that

focuses
on
states
based
definition
of
generative
models
for
example
statistical
models
or
cognitive
models
of
agents
why
it's
important
is
that
with
that
kind
of
a
controlled
structural
vocabulary
for
generative
models
it's
possible
to
reproducibly
and

at
high
throughput
engage
with
a
whole
sequence
of
different
kinds
of
functionalities
ranging
from
reproducible
environment
setup
and
testing
type
checking
resource
estimation
exporting
visualization
accessibility
to
MCP
ontological
analysis
rendering
execution
LLMs
and
beyond
and
more

so
that's
pretty
cool
and
important
core
concepts
are
woven
throughout
and
the goals
are
for
us
as
always
on
these
kinds
of
streams
and
at
the
institute
where
everyone's
welcome
to
find out
more
and
participate
is
to
increase

the
accessibility
rigor
and
applicability
of
active
inference
and
so
this
is
a
triple
play
on
that
triple
play
how
does
GNN
allow
for
clear
textual
specification
of
complex
models
so
we
can
look
at
what
it
looks

like
now
but
with
more
advanced
automated
code
generation
for
example
the
autogenlib
may be
one
library
to look
into
as
well
as
kit
so
kit
is
a
code
synthesis
library
so
right now
this is
like a
v0.1
or a
v1
version
of

the
GNN
spec
it
isn't
that it
has to
be
written
this way
with
this
section
title
it's
just
to
give
an
idea
and
then
we
can
use
more
advanced
code
templating
methods
to
generate
these
from
possibly
even
a
more

distilled
condensed
version
that
renders
into
a
GNN
that
then
gets
used
but
that's
kind
of like
the
assembly
code
question
so
how
does
GNN
allow
for
clear
textual
specification
of
complex
models
well
by
separating
out
a few
key

steps
separating
out
first
in the
state
space
block
the
sheer
declaration
of
the
size
of
size
and
type
of
different
variables
so
c
is
going
to
be
length
three
vector
float
or
you
could
say
well
I'm
going to

be able
to do
my
c
vector
with
integers
then
you
could
say
it's
an
int
and
then
in
the
resource
and
checking
you
could
know
that
you
could
work
with
integer
calculations
so
that's
the
state
space
block
then

connections
which is
a little
bit
generic
of
course
these
need
to
be
nuanced
a
lot
to
be
used
as
a
base
graph
but
the
information
is
there
after
the
state
space
block
and
the
structural
specification
is
the
initial

parameterization

so

here

it

could

be

done

in

a

more

pythonic

fashion

or

it

can

be

just

fully

specified

out

what

is

the

initial

state

or

a

given

kind

of

renderer

could

have

a

default

like

if

something

is
not
provided
with
initial
state
it's
defaulted
to
uniform
or
to
the
identity
matrix
so
whatever
kind
of
rendering
rules
you
want
and
then
you
separate
out
the
ontology
annotation
so
you've
already
basically
initialized
the space
of the

model
and
initialized
the values
and now
you have
this
annotation
layer
which
could
easily
be
swapped
out
with
a
different
language
okay
so
that's
kind
of
a
2023
brought
into
current
year
plain
text
version
of
what
a
minimal
GNN

looks
like
which
is
just
to
focus
on
some
free
form
contextual
information
about
the
generative
model
state
space
for
initialization
with
initial
parameterization
and
the
ontology
annotation
and we've
been having
some great
convos
in the
multi-agent
meetings
about
what
space

does
that
leave
open
then
for
the
orchestration
and the
message
passing
and the
communication
and all
these
kinds
of
logistics
and
operations
of
generative
models
but
that's
where
actually
again
the
case
system
comes
into
play
because
if
the
generative

model
let's
just say
is
the
ingredients
like
we have
lentils
mustard
and
vinegar
and
then
there's
a
sentence
in
the
recipe
which
is
like
the
orchestration
specification
like
with
the
mustard
in
the
kitchen
put
the
sauce
on
the

lentils
and
then
being
able
to
look
at
all
those
prepositions
and
understand
which
of
the
cases
are
each
of
the
models
being
used
in
oh
it's
with
the
mustard
but
it's
on
the
lentils
but
then
if

lentil
has
multiple
methods
or
multiple
different
characteristics
and
it
turns
out
like
with
the
lentils
is
really
hard
on
the
GPUs
because
they're
heavy
but
putting
something
on
the
lentils
doesn't
mean
that
we
need
GPUs
at

all
so
that's
where
we
get
case
specific
resource
estimation
through
this
kind
of
metalinguistic
intelligence
layer
key
syntax
and
structural
elements
just
went
through
them
okay
triple
play
with
GNN
and
and
no
I'm
not
just
talking

about
the
ever
present
comprehensive
who's
on
first
style
dialogue
between
professor
Phineas
Cogswell
and
Pip
Squeaky
Wheeler
as they
discuss
the
GNN
format
I'm
not
just
talking
about
that
triple
play
I'm
talking
about
the
representation
triple
play

which
the
LLM
invented
another
triple
play
of
understanding
conversion
execution
but
it works
well
too
so
say
levy
okay
showcasing
GNN
action
practical
examples
of
the
triple
play
so
let
us
do
that
and
also
see
a little
bit

about
how
cursor
does
this
kind
of
low
key
high
throughput
high
efficacy
program
synthesis
so
let's
make a
new
example
and
work
within
the
framework
we
already
have
and
then
we
can
explore
some
more
far
out
things

so
write
a
new
example
dnn
file
fully
compliant
like
these
two
for
a
pomdp
where
the
ontology
assertions
are in
Spanish
terms
there
are
more
sub
matrices
for
each
variable
but
still
compatible
with
just
these
are
just

kind
of
reminder
notes
but
in
a
better
world
which
we
may
already
be
in
these
don't
even
matter
but
still
compatible
with
visualization
methods
and
the
situation
is that
it
is a
trading
algorithm
being
run
on
an
airplane

where
if
it
loses
money
then
the
plane
runs
out
of
gas
okay
so
with
all
this
kind
of
documentation
and
context
and
especially
templates
and
examples
in the
niche
then
Gemini
or
whomever
you
call
will
be able
to

make
a new
example
so
let us
see
we're
seeing
into
that
chain
of
thought
that
inner
dialogue
that
Gemini
is having
with
itself
and
it's
starting
to
think
through
okay
so
here's
what
we
have
to
do
this
kind
of

pseudo
thought
that's
happening
and
then
it's
writing
out
all
these
details
so
it'll
pop
up
a
file
soon
okay
meanwhile
so
welcome
Andrew
I'm
going to
copy
in
your
question
so
now
we're in
the game
let's
just get
a few
interesting

things
done
during
the rest
of
the
stream
okay
Andrew
wrote
could
a
12
involve
putting
together
some
dashboard
of
the
visualizations
and or
metric
reporting
or
some
synthesis
between
LLM
stage
and
visualization
sure
let's
do
let's
make
one

new
folder
called
site
this
this
I'm
sure
could
be
done
automatically
too
but
I'm
just
going
to
manually
make
an
mcp.py
manually
bring
in
the
init
then
okay
stop
thinking
continue
most
used
command
right
okay
started

up a
new
chat
agent
in
cursor
over
here
so
I'm
going
to
drag
in
main
readme
and
site
and
say
write
a
12
underscore
file
in
src
again
just
dragging
it
in
for
explicit
reference
for
12
site

dot
py
which
will
call
on
all
the
methods
in
site
folder
make
them
all
comprehensively
for
making
a
website
single
HTML
file
with
different
sections
which
reflects
the
output
folder
e.g.
embedding
all
the
digitalizations
and
LLM

outputs
and
etc.
okay
we're
still
deep
in the
thought
chain
okay
we're
in the
Spanish
speaking
airplane
trading
palm
DP
example
so
it's
kind
of
funny
and
I guess
apt
but
for
those
who
have
looked
at
the
2022
active

inference
textbook
this
is
kind
of
like
the
chapter
six
approach
and
it's
also
the
same
thing
that
for
years
we've
been
working
on
in
terms
of
just
like
what
is
the
recipe
for
cognitive
modeling
like
wow

it
sounds
really
cool
about
doing
all
this
active
inference
work
but
how
do
we
really
go
from
there
being
a
situation
of
interest
and
then
whether
for
whimsy
and
fun
or
doing
something
like
an
analyst
would

how
do
you
really
go
from
just
a
system
of
interest
to
having
a
cognitive
model
in
hand
and
chapter
six
from
the
2022
textbook
took
that
part
of
the
way
and
it
gave
like
a
sort
of

proto
recipe
but
this
is
bringing
it
into
another
level
of
structural
possibilities
and
not
to
constrain
either
more
like
a
playground
where
there's
all
kinds
of
fun
jumps
and
pull-ups
and
swings
you
can
do
because
your

expressivity

is

higher

with

these

kinds

of

structures

and

tools

if

you

know

what's

up

so

airplane

trading

on

the

palm

DP

so

as

per

requested

we

have

the

Spanish

language

ontological

assertions

but

the

whole

point

is

it
doesn't
change
anything
with
what
the
state
space
block
or
any
of
the
initial
parameterization
is
so
let's
accept
that
let's
clear
the
terminal
delete
the
output
folder
just
to be
sure
and
then
rerun
python3
main
so

in an
ideal
setting
which
we do
not
know
yet
if
we
are
in
or
not
we
will
have
a
seamless
well
logged
step
through
of
the
new
GNN
trading
airplane
trading
palm
DP
it
should
just
work
so
we'll

see
if
we
get
there
but
bigger
picture
we know
that we
can
get
there
we're
going
to
see
will
it
make
the
visualizations
that
we
want
so
here
comes
this
is
okay
here's
the
airplane
trading
yeah
so
now

we
have
the
we
have
the
variables
from
the
airplane
with
their
Spanish
ontological
assertions
being
rendered
so
it's
like
single
prompt
you
don't
need
to
know
about
the
details
you
can
go
in
there
and
you
can

do
the
engineering
and
set
the
specific
state
spaces
oh
no
no
no
no
we
wanted
it
to be
six
not
three
or
whatever
it
happens
to be
but
again
big
picture
we
can
render
simulations
for
all
of
these

generative
models
to
our
hearts
to
light
so
will
this
script
run
!
I mean
again
we can
sort of
find out
did it
run?
what
happens?
yeah
not sure
if the
run
script
worked
perfectly
but
again
it's
just
a
question
of
okay
what

model
class
is
rendered
what
way
and
then
expanding
that
out
so
that's
one
key
feature
to
demonstrate
which
is
just
purely
within
cursor
let
alone
using
more
more
sophisticated
methods
we
can
do
one
shot
program
synthesis

or
really
like
meta
notation
synthesis
and
use
that
to
just
play
into
the
same
well
typed
pathways
so
that
we
can
make
a
Spanish
localized
simulation
about
doing
trading
on an
airplane
right
off
the
manually
write
all

those
state
spaces
so
I'll
just
drop
that
one
into
archive
okay
so
okay
so
that
was
an
example
of
making
a new
generative
model
so
we
are
again
like
update
your
priors
if
you're
not
here
and
seeing

this
we
have
single
shot
chat
mediated
capacity
to
write
certain
structures
of
generative
model
and
then
there's
a lot
of
work
to
be
done
so
that
increasingly
general
styles
of
GNN
can
be
parsed
and rendered
and so
on
but

that's
actually
why
the
whole
suite
is
structured
as
it
is
because
I
suspect
that
if
we
confirm
which
version
of
GNN
we're
working
in
set
up
our
workspace
run
the
tests
do
the
static
checking
export
and

confirm
that
the
export
was
successful
visualize
it
and
then
execute
and
large
language
model
it
we're
going
to be
in
a
great
spot
okay
so
Tatia
Zot
I'm going
to read
your comment
you wrote
my best
friend
thank you
can you
prepare
group
discord

discord
page
your
youtube
channel
I
want
to
join
you
yes
please
go to
discord
dot
active
inference
dot
institute
and
you'll
you'll
find
the discord
link
so
for those
who are
in the
live chat
now
I'm
just
I just
went to
welcome
dot
active

inference
dot
institute
that's
like the
landing page
if you
want to
get
involved
we do
have
a
discord
and
you're
totally
welcome
to
join
in
and
I'll
paste
the
link
just
directly
into
the
youtube
live
chat
as
well
and
that's
one

place
activity
happens
but it
happens
wherever
you are
there
you
are
so
here
we
go
with
the
demonstrating
a sort
of
package
level
flexibility
in terms
of being
able to
bring
in
a
new
functionality
into
the
GNN
space
so
here
we
have

the
site
folder
coming
together
and
then
it
will
make
a
12
underscore
site
dot
py
and
then
it
will
add
that
in
to
main
so
that
it
runs
12
underscore
site
so
that
when
we
add
a

new
example
in
and
we
run
it
all
with
main
that
will
have
it
all
pump
up
so
let's
do
another
example
so
here
I've
created
src
site
mcp
so
this
is
like
a
way
to
confirm
yeah

these
are
the
methods
related
to
the
website
like
generate
pipeline
summary
site
that's
going
to be
exposed
through
mcp
it's
not
fully
functional
yet
but
it's
all
there
so
that
people
could
call
sequential
mcp
calls
and
then

say
like
here's
the
recipe
you
blend
up
a
then
you
add
b
take
two
c
and
combine
them
with
d
then
do
all
this
and
do
that
and
then
that
so
that
level
of
programmatic
synthesis
is

something
that
is
very
excitingly
on
offer
from
dspy
which
is
a
programming
structure
it'll
look a
little
cleaner
in the
repo
for
doing
program
synthesis
at the
level
of
llms
which
is
where
we're
going
to
have
the
mcp
app

so
dspy
is
going
to
be
useful
for
programming
sequences
of
mcp
type
calls
kit
or
something
similar
is
going
to
allow
program
synthesis
and
code
base
level
intelligence
mcp
whether
it's
fast
mcp
or
another
method
is

going
to
support
making
these
tools
available
even
if
somebody
doesn't
have
the
whole
repo
because
we
can
offer
an
endpoint
as
earlier
mentioned
cerebrum
and this
sort of
lexical
intelligence
paradigm
is going
to help
us make
case
specific
renderings
of
generative

model
so that
when we
understand
from the
beginning
what
different
roles
a
given
object
or
generative
model
is going
to
serve
we're
going
to be
able
to
understand
what
kinds
of
rendering
pipeline
are
needed
so
again
if we're
talking
about
a
table

are we
talking
about
placing
something
on
the
table
or
are we
talking
about
hitting
a nail
with
the
table
because
if we're
just
going
to
place
something
on
the
table
we only
need
enough
RAM
to
pick
up
the
object
that
we're

putting
on
the
table
like
we're
going
to
put
the
table
on
the
table
but
if
we're
going
to
put
the
table
on
the
pen
then
we
need
something
like
a
GPU
that
can
pick
up
a
table

so
that's
how
prepositions
are
and
what
they
signal
and
then
there's
just
again
these
folders
can
have
more
documents
and
more
different
frameworks
brought
in
X402
which
we were
talking
about
yesterday
is
just
thinking
okay
how
could

we
do
micropayments
so
that
if
somebody
thinks
I'm
going
to
ring
up
the
institute
and
ask
for
a
textbook
written
in
this
language
and
then
we
do
that
on
the
back
end
or
I'm
going
to
have

a
generative
model
written
up
like
this
then
there
can
be
small
charges
or
just
kind
of
authentication
different
kinds
of
methods
to
offer
these
up
as
services
okay
the
pipeline
12
is
integrated
so
let's
see
if

the
main
can
handle
this
running
so
just
clear
I'm
going
to
delete
the
output
folder
that's
why
the
output
folder
is
standalone
because
how
satisfying
to have
SRC
and
dock
that's
it
and
this
is
like
the
minimal

dual
channel
strategy
basically
dock
is
suggestive
that
it's
going
to
be
more
natural
language
like
SRC
is
suggestive
that
it's
going
to
host
all
of
the
code
and
it
is
that
way
and
then
we
have
a few

top
level
scripts
so
we
have
the
citation
if
anyone
sees
relevance
in the
work
partially
filled
I'll
change
the
release
date
to
today
per
version
0.1.0
code
of conduct
auto
written
but we
can tweak
that
too
contributions
already
even
those

just
listening
or
watching
or
reviewing
summaries
thereof
it's
people
and
agents
like
you
that
make
GNN
such
a
great
tool
it's
so
true
license
MIT
license
for now
those
with
more
insight
into
how
we
can
have
more

advanced
and
multi-channel
licenses
get in
touch
etc
but
that's
where
it's
at
for
now
and
as
far
as
I
understand
it's
the
first
program
synthesis
based
generative
model
creation
assistance
tooling
framework
so
I
hope
that
people
who

find
value
in
that
can
get
in
touch
maybe
support
the
institute
maybe
support
the
project
maybe
use
the
framework
the
readme
is
we
have
a
cyber
physical
and
cognitive
security
policy
in
this
GNN
project
so
inquire

within
for
more
and
we
do
want
to
try
to
be
supportive
so
whether
it's
during
this
live
chat
and
you
have
questions
you
want
to
ask
or
whether
you
have
github
issues
that
you
want
to
open

up
or
other
questions
about
like
what
can
we
do
with
this
how
can
I
etc
then
we're
there
all of
the
extraneous
documentation
as well
as
different
other
ideas
like
how
could
this
be
useful
for
multi-agent
simulations
like

well
maybe
we
expand
the
GNN
structure
to
include
an
agent
block
and
then
we
have
a
GNN
file
that
is
the
notation
for
a
multi-agent
simulation
and
it
says
you're
going
to
use
one
instance
of
five

instances
of
this
GNN
and
one
instance
of
that
GNN
and
so
we
can
make
structured
notational
fields
for
doing
all
that
kind
of
simulation
level
reproducibility
and
communication
so
that
and
then
we
can
have
either
kind

of
state
spaces
that
are
wired
up
again
separate
from
their
English
name
so
we
can
flip
in
and
out
of
using
that
state
space
representation
where
the
variables
could
be
known
like
by
their
cryptographic
hash
all the

way
on
through
just
okay
now
you
want
to
turn
it
into
this
language
lens
so
just
ideas
about
how
we
could
use
GNN
and
structured
fields
for
the
communication
questions
and
keeping
with
this
spirit
and
inspiration

that
it's
implementation
agnostic
GNN
isn't
the
rendered
version
that's
executable
GNN
is
the
notation
notation
generalization
so
if
a given
kind of
simulation
level
or agent
level
description
is written
in GNN
like DNA
like source
code
then
you may
have
a rendering
element
that
is

able
to
render
and
transform
the
contents
into
the
format
you
want
that's
docs
okay
pipeline
has
finished
it
says
successful
11
one
success
with a
warning
so
let's
see
what
the
site
okay
looks
like
12
thinks
that the

site
was
created
so
let's
find
out
okay
here's
the
summary
site
moment
of
truth
wouldn't
say
it's
the
most
advanced
site
that I've
ever
seen
but
it
does
the
job
it
has
all
the
information
and
then
that's

how you
develop
modules
just
like
okay
now
develop
site
to be
like
this
so
here
I'll
just
remind
it
of
that
ensure
the
readme
includes
site
and
the
12th
script
make
any
improvements
within
the
site
folder
ensuring
you

use
separate
scripts
within
site
if
it
is
getting
like
long
modular
methods
okay
clear
so
it
will
take
some
some
work
some
collaboration
from
hopefully
all
to
many
of
us
that
will
make
it
more
rigorous
for

example
having
a
strict
template
of
a
certain
kind
like
hey
if
you
want
to
do
discrete
time
discrete
state
spaces
and
get
it
to
land
the
plane
in
pyMDP
here
is
the
standard
example
or
here's
the

code
template
the
code
generation
template
the
kind
of
stuff
we
can
do
with
code
generation
libraries
here's
the
code
generation
library
format
schema
for
if
you
want
to
make
sure
that
it's
going
to
go
into
pyMDP

well
like
this
should
be
like
this
and
you
need
this
you
can't
have
that
etc
let's
add a little
functionality
with
rxinfer
so
while the
website
stuff
is
happening
assess
and
use
at
web
search
to
understand
how
rxinfer
at

model
statements
are
used
e.g.
e.g.
the
generative
model
joint
distributions
in
format
where
it
can be
parsed
by
rxinfer
into
a
executable
model
then
make
those
improvements
to
so
that
here's
the
rendering
so
that
rendered
outputs
are

accurately
conveying
all the
information
make
an
rxinfer
dot
or
utils
dot
py
or
rxinfer
utils
dot
jl
if
needed
yeah
the
gemini
2.5
sometimes
stalls
out
kind of
randomly
we'll
just say
alright
I'm gonna
make this
edit
and then
just
hangs
there

but
generally
speaking
it has
been
great
okay
let's
add
another
example
see if
we can
get into
some
other
kind of
model
that we
okay
so we have
website
chugging
away here
rxinfer
improvement
here
now
we're
gonna
write
another
example
according
to
these
compliant
examples

write
another
kind
of
generative
model
not
a
pomdp
that
will
creatively
and
functionally
demonstrate
important
features
and
capacities
of
package
make
the
situation
something
esoteric
yet
tangible
semicolon
available
though
ineffable
make
it
best
best
best
possible

okay
um
it's
been
one
hour
if
some
super
awesome
funny
stuff
happens
I might
stay
on
but
otherwise
just
a
few
more
minutes
because
I
think
I
have
pretty
much
conveyed
where
it's
at
and
some
of
the

core
features
so
in
the
last
section
here
like
if
anyone
has
an
idea
for
an
example
or
another
function
that
they
want
to
see
or
they
have
a
specific
question
that
they
would
like
addressed
in
this

stream
please
write
it
in
the
next

few
minutes
okay
Okay.

Greetings, Andres.

Oh, now that could be a fun plot twist to include some of your work into this.

Okay. I'll make a just a sort of starter pack, NTQR, and a NTQR, or GNN, NTQR, MD.

Okay. I'll just leave that there for now, but I'm sure that you and others could imagine a few different ways that NTQR comes into play.

It could be like a module offered in a multi-agent simulation setting.

Like, just as another module, you could run on top or after the multi-agent.

Then you can run it with this NTQR layer.

Just trying to delete this, but it's a slow computer.

Okay.

Okay. So this is what the esoteric...

Oh!

Goodbye, cursor.

It was good to know you.

Pull quit.

Okay. We're back.

Okay.

All right.

So it's a GNN.

Bayesian network that stochastically generates a verse concept.

It aims to model the flow of poetic inspiration from emotional and elemental inputs to concrete poetic characteristics.

So pretty cool.

This is getting us into the space of explicit active inference language models.

So not just using the inputs and outputs of language models and treating those or conditioning those with active inference,

but this is actually getting into what we can now do.

So now make another example.

Absolutely comprehensive and vast.

called an active inference language model that has state spaces for e.g. sound.

phoneme,

syntax,

morphosyntax,

semantics,

situational,

narrative,

all the nested and interacting

levels

so we can really,

truly,

actually,

finally,

seriously,

safely,

and

beneficially

have a full GNN

based

model of language.

language.

No training

data sets

here.

Just one shotting

the triple play

based upon the GNN

launchpad

for making

the world's

first actual

active

inference

linguistic

intelligence

agent

pool box.

Okay.

Have it continue with RxInfer

because it got

quit on

and

looks like the website one did too.

we'll just say continue

and then we'll run it again in a few seconds.

Yeah,

also the new

version of

cursor

.50

it has some

background.

Well,

one important thing,

indexing was broken out into its own

separate tab.

If you've watched a bunch of

InferAnt streams,

you know that I was always going into

like features

and then scrolling down to the code base

and doing the resync.

There's

MCP,

but this is exactly how easy it's going to be

to then have

all this GNN stuff

tucked away

and then

point the MCP server

at it.

So then

other people
or agents
will be using cursor
or whatever
and then it'll be like
plan,
make a plan
and then that could invoke
this program synthesis,
rendering,
type checking,
visualization,
simulation,
execution,
multiple environments,
logging,
et cetera,
et cetera,
et cetera,
website generation.
And also there's the
background agents.
I don't know where
it is in the settings.
I haven't looked at it,
but there's a background agent
that you can
have.
So that could be very useful.
For example,
a background agent could say,
always
absolutely
vigilantly
be assessing
if everything
is totally good
and up to mega standards

for everything we're doing
and just let us know
if not,
but don't make changes
by yourself.

Okay.

AILM.

Phonetic

target

and articulation.

let's see what this looks like.

So again,

if this were using motifs

and the structure

that we knew

was going to work,

then like

we would know

it would work.

Okay.

Let's close that chat.

See if the website is done.

All right.

Here we go.

We're going to

throw caution to the wind

and see if this

active inference language model

will run in PyMDP.

It's not going to.

Well,

let's delete the output

just to confirm.

Delete the output folder.

Okay.

So let's just see

how it broke it up.

See how the LLM

broke up
this structured analysis.
And this is just
a total,
total
early
hint
of the power
of using
language models
and general purpose
agents
to write
structured texts
which we then
can bring into
this explicit
program execution
framework.

Okay.

So hidden state factors,
phonetic target
and articulation.

Then there's a
lexical
morphosyntactic level.

So
lexical concept ID
that would be like
the diction
like table chair.

Then there's the
current syntactic rule.

This takes us right back
to cerebrum
and the different
grammatical cases.

Like

is the table

acting?

Is the table

the recipient

of action?

Is something

being placed

on the table?

Is something

being placed

near of the table?

And then there's

tense,

number,

gender,

etc.

That brings us

from the sort of

lexical

into the more

properly fragments

of semantics

which

here,

let's see.

Semantic proposition.

So propositional

avalanced

propositional

assertion-based

semantic representation.

not the only one,

but

that's why we can

make multiple

kinds of GNN files.

We can say,

make a new GNN file

that explores
a really,
really different
and radically
contrasting
semantic concept.

Narrative
context,
internal goals
and predictions.

Observation
modalities.

you can make,
no,

no,
we only want
it to observe
auditory.

We want to have
confirmation
that the observations
are only coming
in through sound
and that all
the semantics
is being
endogenously
generated.

or we want
the policy
output to be
red,

yellow,
green,

just three states.

Or we want
the policy

output to be
words in the
dictionary.

Then we would
have literally
next token
prediction,
policy prediction
in this kind
of AI LM
would be
just like it is
in transformers.

So this
state space
block is
obviously
massive.

the
connections.

Let's see
if the
visualization
went through.

Okay,
here's the
poetic muse
model.

Interesting.

Here's the
poetry
model.

Ontology
assertions
in the state
space.

That one
was one-shotted.

And then
let's look at
the language
model.

Okay.

Looks like
it has some
differences
so that we
could get
to this
state space
representation

but that

the,

but like,

it could be
a breaking
change but

the kind
of suggestion
would be
dragging this
in.

Hey,

we didn't
get all the
visualizations
that we
thought of.

Ensure

that

visualizations
are generated
properly

for this.

And then

it would be

off to the
races again.

Exports.

Here's its
attempt to

export it

in

JSON

style.

So maybe

you only need

to use,

maybe when you're

calling MCP

or you're

using this

for your own

research,

maybe all you

want to do

is specify

it,

type check

it,

and then

export it.

You don't even

want to visualize

it.

Or maybe you

just,

you don't care

if it's

type checked,

you just

want to

visualize it.

So that's

why these
methods are
written out
separately.

This is
the
language model
failing to be
put into
PyMDP
format.

Let's see if
it gets a
little bit
better.

And
similarly,
RxInfer
didn't
continue
through.

Comprehensibly
ensure that
at model
blocks
really
are
output
to
there
given
the
GNN
input.

So maybe
I'll
run one
more.

Okay.

refactor

the

website.

All right.

So we'll

let this

RxInfer

update

come through.

And the

LLM

when I just

interrupted

it.

That

would be

So here

we get

that

LLM

interpretation

generated

from an

OpenAI

call at

this point

to describe

the model

and to

answer

different

questions

about the

model.

So you

can have a

default

question

kit,

Q-kit,

ask these

different

questions.

Let's

see what

happened when

it was

tried to

execute it.

Yep.

A matrix

must be a

numpy array.

At least

it's a real

error.

Okay.

So where

to go is

going to

depend on

where you're

at,

what time

it is,

all that

kind of

stuff.

I hope

that people

look through,

contribute,

improve.

I think

one of the

most high
yield
payoffs,
if somebody
is familiar
with structured
code generation
methods or
type checking
or program
synthesis
techniques
would be
to develop
a
template,
not to
fix it as
like the
one and only
template to
use,
but just
here's a
template that
we can
reference all
of the other
methods against
and also
know which
degrees of
freedom we're
going to be
able to
move with
and break
because right

now,
maybe we'll
say,
oh,
well,
you know,
the visualization
wanted there to
be a hyphen
here and then
you add the
hyphen here and
the visualization
works,
but then the
other,
you know,
something else
doesn't work.
So that is
where I believe
that using
this order
of operations
is going to
be helpful
or something
similar to
it,
of course,
pending
everyone's
input,
but basically
like first
verify that
the GNN
module is

loaded up
and use
this file
structure and
punctuation
to confirm
you're like,
yeah,
this is the
version of
GNN we're
working with,
this is what
the symbols
mean.

Then,
but that
should just
output the
documentation and
the version.

It shouldn't
need any
requirements to
be installed in
the workspace.

Then,
setup,
also it's kind
of standalone
and simple,
but it should
install the
requirements that
are needed.

Then an
extremely
comprehensive

test suite
can be run,
including tests
about later
modules.

So maybe
there's a
module that's
not used in
your flow,
but still
it is
type
checked
and tested
with a
template.

The type
checking is
basically the
first of the
real ingress
steps,
and this is
where I know
that there are
better methods
that people can
bring to the
table.

Okay,
let's run
main one
more time
just to see
if it can
get this
rx and

for plane
landed.
Okay,
so GNN
type checker,
maybe the
resource
estimator
could be
broken out
to its
own
script
or folder
even,
but the
resource
estimation
is not
costly,
so I
figured it
probably can
stay with
the type
checking
because it
is based
upon the
inferences
made by
the static
type
checker
engine.
Export,
that could
go slightly

earlier
because you
don't need
to type
check,
but type
checking
tells you
you're going
to have an
issue with
this variable,
it's not
stated properly,
or you
won't,
and then
export kind
of confirms
that,
that you
can read
in and
out.
So if
everything is
green light
up on
through export,
the goal
is like,
you should
be in
the clear
because we've
already done
static testing,
we've already

demonstrated that
we can read
in and then
read out
a given
file.

So like
downstream
of five,
I think
it may
even use
the re-exported
form.

after having
already been
checked to
be the
same as
the input.

So it's
like if we
can regenerate
the original
GNN file
100%,
then we
may want
to work
with that
air-gapped
version
purely.

Because on
one hand,
it's like it
would be the
same, but

on the other
hand, it's
a version
that we
know that
we wrote.
visualization,
there could
be generic
and specific
visualization
methods, and
so that
would require
a lot of
flagging,
like where
are we
doing heat
maps, where
are we doing
other kinds
of visualizations.
MCP, again,
that could be
moved to a
different order,
but it should
expose these
methods and
the
neurosymbolic
possibilities for
MCP use.
Like, imagine
the so-called
language models,
the transformer

LLMs,
calling the
real language
model as
program
synthesis,
and then
that
active
inference
probabilistic
language model
could have
a GNN
dialect, so
it could do
program
synthesis on
GNN
within a
linguistic
GNN
representation
inside of
a cookie
of just
probabilistic
general
inference.
Ontology,
that one
could just
be more
of an
archival
ladder,
note.
It doesn't

have to be
eight, but
it's pretty
interesting.
Like, if we
wanted to
say, hey,
this
undergraduate
course or
this
micro
certification
or training,
you are going
to fully
understand this
drone
forest
simulation
example.

It's a
multi-agent
simulation with
drones in a
forest.

Here's the
file that we're
going to be
looking at,
and everything
is going to
be about
those ontology
terms.

So,
every single
thing we

say is
going to
be semantically
laser-eye
on your
education
related to
drones in
the forest.

And,
we can
have one
foot in
that content
and still
pivot out
and associate
and talk
about other
topics,
but let's
be clear
with which
ontology
terms we're
using in a
controlled
fashion.

Render?

Maybe
another word
is slightly
preferable,
but this
is just
preparing
the simulation
for execution.

So,
having a
render,
like a
prepare,
and then
the execution
phase,
may just
be as
simple,
provably,
as
Python
and then
the Python
script name.
That would
be ideal.
Doesn't do
any other
stuff.
if it
just
pulls the
trigger
on what
render
loaded up.
LLM
again.
Here,
you can
just say,
make it
so that
LLM
does this

other
kind of
thing.
That's
just more
general
LLM
tools,
and then
today on
the stream,
we made
this
site
to generate
it.
So,
let's,
in the
closing bit,
okay,
let us
look at
the
RxInfer,
to see
if we
got a
better
RxInfer
simulation.
Okay,
that one
didn't.
That one
didn't.
Okay,
so,

program
synthesis
for RxInfer
waits till
another day,
but one
thing that's
going to be
awesome with
RxInfer
specifically
is another
degree of
freedom where
we will be
able to
specify
RxInfer
models,
and then
we can
either run
that locally
or dispatch
that to
the RxInfer
server API.
So,
then,
that's like,
earlier I
said you can
write recipes
that you can't
cook,
but this is
like separating
out,

now it's like
you can write
a recipe that
you know you
can order,
because you
can fully
specify it
and then
send it to
another
endpoint
that's just
an RxInfer
server that
will run
your app
model with
your conditions,
but you've
already done
the type
checking and
the verification,
you know that
you're working
off of a
generic
implementation
agnostic
representation.

Okay,
and the
LLM is
still
processing.
processing.
So,

let us,
let that
finish then
pull in the
site,
but in
terms of
the last
content
here,
we will
read a
little bit
from the
plain text
to triple
play.
So,
we're going
to improve
this with
one prompt
and then
see if
anybody has
any comments
or questions
after reading
it.
We'll
write a
new
Who's
on
First
dialogue.
Triple
play

style.
Write a
new
novel
dialogue
Who's
on
First
style
whimsical
and deadly
serious
total
americana
deep
lore
absolutely
conveying
all the
topics
with
quote
triple
play
puns
related
to
dragging
the
doc
folder
in
such
that
through
the
reading
!

SRC code

becomes

strangely

to those

who pay

attention.

Again,

this must be

absurd,

speculative

realism for a variety of reasons, delivered in PhD dissertation length,

and

the

!

and

the

truth

is

strangely obvious to those who pay attention.

Again, this must be like absurd, speculative realism for a variety of reasons, delivered in PhD dissertation length,

delivered in PhD dissertation length, who's on first dialogue style, or it could be three conversants, if that will make the triple play, pun play, all the better.

Thank you, Andrew P.

Okay.

Okay.

We could have like a, we could do a benchmark on how much do each of these LLMs know about 1920s baseball more.

Okay.

While that chain of thought proceeds, let's just see the LLM stuff.

So it's written four of the five.

So each, each, um, currently, in the GNN folder examples is the target input directory.

We might want to have another directory called input, or something, so that it's just obvious which files are

going to be, uh, iterated through.

But obviously, that's one of the, um, ways to work faster.

If you only have one, uh, file in here, like, it's going to happen way faster.

But right now it has to do all the iterating and looping for all five of these examples.

So I think I'll move...

I'll move three of them into archive.

Just so that people can get started and get it running faster.

But the language, they all could be developed in versions in really fun ways.

Okay.

Okay.

Okay.

we'll generate this dialogue and then if there's any last comments

let's just see what else is in the docs folder about GNN this just summarizes that's a starting

place here's a domain specific language DSL spec sheet for it but more human legible

examples breaking down the different examples file structure these kinds of markdown representations

mermaid plots super useful how does it get implemented again the goal and the whole

vision is like specifying generative models at a level above implementation so if you're on a

Intel versus an AMD processor the program is still going to work so similarly like if you're gonna be

there might be some features that are easier or only available within one of these computer languages or

within one of these implementation environments but in principle we're talking about

specifying something at a more semantic level as mentioned and alluded to a little bit I think

that this kind of executable program synthesis is going to have a lot of relevance for

neurosymbolic synthetic hybrid intelligence architectures there's a whole world of

multi-agent specification and ways that we can work at this layer

okay Gemini one more chance

really fully write out this conversation which we need

uh here's the plain text or a summary of it of the paper itself but it's linked in the in the repo with

Jakob some information on the syntax but this could be reflected probably better using this type of kit

concept

concept

uh

no

this is the repo

paste slash kit

code-based mapping symbol extraction code search building LLM powered developer tools agents and workflow when I saw that I was like

yes

and making MCP servers access to it

uh

that's basically the the docs just to review what's there now

x402 microservices

rx infer we can make a pymdp

folder just to be

leaving a stub there

and that could have like more generic documentation information on doing

gnn pymdp

but it'll just be

do b

let's see

all right

here we go

all right i'm gonna push it on github

read this uh one-shotted dialogue

oh the LLM is still

working for some reason

okay

uh read the dialogue and then either it will uh

either it will end on an extremely dramatic

funny note we haven't looked at it yet

or maybe somebody will have an appropriate comment or question in the next minute or two

at which point I will then read that one

so here we are in other

triple play dialogue

okay

here we go

the GNN triple play

a dialogue in three fits and many errors

characters

abner doubleday

earnest

precise

a true believer in the system

capitalized

he's trying to explain the rules of generalized notation notation

GNN

he often clutches a heavily annotated copy of what looks like a rule book but is in fact
doc slash gnn file structure doc dot md
casey specs stumble
an old-time baseball manager
weathered
full of cracker barrel philosophy and prone to spectacular misunderstandings
he's always looking for the angle or the specs on the new talent and how this all plays out on the diamond
which he sometimes confuses with the GNN connections graph
he squints at everything through thick smudged spectacles
branch
ricky wright
a shadowy almost spectral figure
perhaps a long disenfranchised league commissioner
a rogue statistician
oh brother
or a philosopher of the great american pastime
now haunting the ontological back alleys of model specification
he speaks in iraq oracular often unsettling pronouncements his voice like the creek of an old stadium gate
setting
the deepest dustiest back office of the national generative model league
NGML
headquarters
a forgotten annex that smells of old leather
stale cigars
lenament
and the faint sharp tang of ozone from ancient calculating machines that were rumored to predict not just
game outcomes but the very fabric of fan belief
yellowed schematics of factor graphs and bay's nets are pinned to the peeling wallpaper
interspersed with faded daguerreotypes of stern-faced men in bowler hats
who might have been pioneering statisticians
spiritualists or perhaps both
a single fly-specked gas lamp sputters erratically
casting long dancing shadows that make the gnn diagrams on the wall seem to shift and writhe
a ticker tape machine in the corner occasionally coughs out cryptic decontextualized messages
e.g. model name not found state space undefined policy does not converge
the curtain rises
or rather the dust moats swirl expectantly
abner is pacing occasionally tapping his rulebook

specks is trying to whittle a piece of what might be a broken bat or a very large pencil with a pen knife
rickety is half hidden in the shadows only a glint of an eye and the occasional wisp of pipe smoke visible
fit the first
who's on model name
abner
clearing his throat adjusting his spectacles
gentlemen
welcome
or rather welcome back to the eternal deliberation
today we must rigorously define the parameters of our
our understanding
it all begins as it musts with the model name
clarity here is paramount
specs
model name eh
is he a new kid
what's he bat
lefty righty
got any pop
we need pop
the fans
they scream for pop
and where's he from
don't tell me he's another
one of them fancy college boys who can't hit a curveball in the state space block
abner
sighs
a small puff of dust rises from his waistcoat
no specs
model name isn't a player
it's
it's the designation
the unique identifier for the entire conceptual architecture we are attempting to instantiate
think of it as the banner under which the whole enterprise marches
the grand title on the spine of the almanac
so like the team name
the generative giants
the notation nationals

the markov chain marauders
i had a team once
the mudville nine
good bunch of fellas
terrible fielders
lost 17 straight in the time horizon
17
each discreet time step of fresh agony
rickety
voice rasps from the shadows
like dry leaves skittering across an abandoned home plate
the name
yes
before the word
there is the naming
before the connections are drawn
the model name is whispered into the void
it is the first casting of the die
the initial prior block from which all subsequent beliefs unfold
does the name sing to the actin-phontology annotation
or does it merely echo
in the empty grandstands of failed hypotheses
precisely mr right
well
almost
it's less about singing and more about
unambiguous referencing
as stipulated in doc slash file structure
doc.md
section 2
it's a descriptor
a label
for example
gnn
pi mdp agent
concise
informative
pi mdp agent

sounds foreign

is he one of them international league fellows

got a good arm

this agent

can he play multiple positions

like can he cover hidden state

and also fill in at observation

if we're in a pinch

versatility

that's the ticket in this league

a real triple play threat

if you catch my drift

abner

massaging his temples

it's

pi mdp refers to the python library for markov decision processes

the agent is the model itself

it's not about positions on a field specs

but about variables in a system

variables eh?

so like guys who are inconsistent?

hot one day, cold the next?

we got a whole roster full of variables

s underscore f0

couldn't hit water if he fell out of a boat last tuesday

and o underscore m1

don't get me started on o underscore m1's fielding errors

cost us the pennant that one

initial parameterization was all wrong for him

rickety

the initial parameterization

the primordial soup

the conditions at the dawn of the simulation

was the back cork from the start?

were the dice loaded in the state space block?

or was it merely

a d underscore f0

set to uniform ignorance

awaiting the harsh lessons of observation?

yes

the d underscore f0 vector

the prior belief over initial states for the factor 0

you see specs?

model name sets the stage

then comes the model annotation

model annotation

is that the fella who writes down what model name does?

like a scorekeeper?

or one of them newspaper scribblers?

always looking for a model name to tear down

we had a guy

Scoop McMurphy

used to say

my policy vector was more like a meandering suggestion

the nerve

apner

model annotation is a free text

caption specs

it explains the model's purpose

its context

its raison d'etre

it's the color commentary

if you will

to the model name's play-by-play

it's where you describe the triple play of GNN

the text-based specification

the graphical model intuition

and the executable cognitive model

triple play

now you're talking abner

so

model name is the batter

model annotation is the guy on first

and

who's on second?

is it state space block?

he sounds like a big fella
a slugger
this is gonna be a long time horizon
state space block isn't a who
it's a what
it's the where
it's the definition of all possible states the system can be in
the dimensions of the diamond itself
i'm gonna skip on
since there's many sections
to the end
but
let's
skip
okay
last section
abner
looks from specs to rickety
a daunting realization
or perhaps just fatigue in his eyes
so
src
slash main dot py
that orchestrates the one GNN parsing
the two setup environment
configuration
the three export transformations
the four GNN type checker validation
and the five render execution
that's just the empire calling the game
ensuring the rules
of generalized notation notation are followed
the empire
of course
that's main dot py
always sticking his nose in
checking the GNN version and flags
and making sure the footer and the signature are in order

thinks he's god or g
or some other fancy letter
but without him
it's all just output
slash log
slash chaos dot txt
rickety
the main dot py
the silent orchestrator
the loom upon which the GNN tapestry is woven
it does not judge the model annotation's poetry
nor the equation's elegance
it merely ensures the pipeline flows
from text to graph to code
a humble servant of the triple play
the true mystery
gentlemen
is not who's on first
or what's on second
or even I don't know on third
it's why we play the game at all
and for that answer
you'll need to consult a model name
far grander than any found in src
slash GNN slash examples
rickety fades further into the shadows
speck starts idly checking model parameters on an imaginary bat
abner stares at his GNN rulebook
a single bead of sweat tracing a path
through the dust on his forehead
the ticker tape machine clatters one last time
simulation complete
observed outcome unlikely under current priors
suggest revising world models
or perhaps the rules of the game itself
the gas lamp sputters and dies
plunging the room into near darkness
illuminated only by the faint residual glow

of unvalidated hypotheses

ail ail ail

All right.

Thanks, everyone.

Hope this was pragmatic and epistemic for you.

Looking forward to people's feedback and collaborations.

Acting for Surf.

Thank you.